

探究八皇后问题

逻辑编程：告诉计算机规则约束，计算机推导。

命令式编程：告诉计算机具体步骤，计算机执行。

八皇后问题：

在 8×8 的棋盘上放置 8 个皇后，使得任意两个皇后都不在同一行、同一列或同一对角线上。找出所有满足这个规则的摆法。

1. 穷举法解题

非逻辑编程：

因为任意两个皇后不能在同一行和列，所以构造一维数组 `col[0,1,2,3,4,5,6,7]` 代表 8 个皇后所在列号。



如图可表示为 `col[0,4,7,5,2,6,1,3]`

行列约束: `[0,1,2,3,4,5,6,7]` 数组，无重复列

对角线约束：“行索引之差”的绝对值不等于“列索引之差”的绝对值

核心思想：

将问题转化为对数字 `[0,1,2,3,4,5,6,7]` 的所有可能排列进行暴力搜索。

每个排列 `col` 的下标代表行号，其值代表该行皇后的列号，天然满足了“不同行”的约束。

实现步骤：

生成全排列：使用递归交换算法（`generate_permutations` 函数）生成所有可能的列位置排列。

过滤有效解：对每一个生成的排列，使用 `is_valid` 函数检查其是否满足对角线约束（即任意两个皇后，其行索引之差的绝对值不等于列索引之差的绝对值）。

伪代码:

```
// 递归函数：生成全排列
函数 generate_permutations(数组, 起始下标, 结果列表)
    if 起始下标 == 数组长度 - 1 则
        将 数组的副本 添加到 结果列表
        返回
    结束 if
    while i:起始下标->数组长度 - 1
        交换 数组[起始下标] <-> 数组[i]
        // 递归调用下一层
        调用 generate_permutations(数组, 起始下标 + 1, 结果列表)
        交换 数组[起始下标] <-> 数组[i]
    结束 while
结束函数

// 检测函数：排列是否满足八皇后对角线约束
函数 is_valid(排列数组)
    设 n = 排列数组的长度
    while i : 0->n - 2
        while j : i + 1->n - 1
            if |排列数组[i] - 排列数组[j]| == |i - j| 则
                返回 假
            结束 if
        结束 while
    结束 while
    返回 真
结束函数

// 主求解函数
函数 solutions(数字列表)
    // 初始化存储列表
    设 所有解 = 空列表
    设 所有排列 = 空列表
    // 生成所有可能的排列
    调用 generate_permutations(数字列表, 0, 所有排列)
    // 过滤出满足对角线约束的排列
    for 每个 排列 在 所有排列 中
        if is_valid(排列) 为真 则
            将 排列 添加到 所有解
        结束 if
    结束 for
    返回 所有解
结束函数
```

Generate_all.py

```
import os

# 创建结果保存目录
result_dir = "generate_out"
os.makedirs(result_dir, exist_ok=True)

def is_valid(list):
    n = len(list)
    for i in range(n):
        for j in range(i + 1, n):
            if abs(list[i] - list[j]) == abs(i - j):
                return False
    return True

# 生成全排列:
def generate_permutations(nums, start_idx, result):
    if start_idx == len(nums) - 1:
        result.append(nums.copy())
        return
    for i in range(start_idx, len(nums)):
        nums[start_idx], nums[i] = nums[i], nums[start_idx]
        generate_permutations(nums, start_idx + 1, result)
        nums[start_idx], nums[i] = nums[i], nums[start_idx]

# 获取结果并保存
if __name__ == "__main__":
    nums = [0, 1, 2, 3, 4, 5, 6, 7]
    all_permutations = []
    generate_permutations(nums, 0, all_permutations)
    # 保存结果到文件
    output_file = os.path.join(result_dir, "all_sequences.txt")
    with open(output_file, "w") as f:
        for idx, solution in enumerate(all_permutations, 1):
            f.write(f"{solution}\n")

    print(f"Results saved to: {output_file}")
    print(f"Total solutions: {len(all_permutations)}")
```

生成所有排列可能

- PS C:\P&T_AI\Exhaustive_non-logical> python generate_all.py
Results saved to: generate_out\all_sequences.txt
Total solutions: 40320

Exhaustive_non-logical > generate_out > ≡ all_sequences.txt

1	[0, 1, 2, 3, 4, 5, 6, 7]
2	[0, 1, 2, 3, 4, 5, 7, 6]
3	[0, 1, 2, 3, 4, 6, 5, 7]
4	[0, 1, 2, 3, 4, 6, 7, 5]
5	[0, 1, 2, 3, 4, 7, 6, 5]
6	[0, 1, 2, 3, 4, 7, 5, 6]
7	[0, 1, 2, 3, 5, 4, 6, 7]
8	[0, 1, 2, 3, 5, 4, 7, 6]
9	[0, 1, 2, 3, 5, 6, 4, 7]
10	[0, 1, 2, 3, 5, 6, 7, 4]
11	[0, 1, 2, 3, 5, 7, 6, 4]
12	[0, 1, 2, 3, 5, 7, 4, 6]
13	[0, 1, 2, 3, 6, 5, 4, 7]
14	[0, 1, 2, 3, 6, 5, 7, 4]
15	[0, 1, 2, 3, 6, 4, 5, 7]
16	[0, 1, 2, 3, 6, 4, 7, 5]
17	[0, 1, 2, 3, 6, 7, 4, 5]
18	[0, 1, 2, 3, 6, 7, 5, 4]
19	[0, 1, 2, 3, 7, 5, 6, 4]
20	[0, 1, 2, 3, 7, 5, 4, 6]
21	[0, 1, 2, 3, 7, 6, 5, 4]
22	[0, 1, 2, 3, 7, 6, 4, 5]
23	[0, 1, 2, 3, 7, 4, 6, 5]
24	[0, 1, 2, 3, 7, 4, 5, 6]
25	[0, 1, 2, 4, 3, 5, 6, 7]
26	[0, 1, 2, 4, 3, 5, 7, 6]

共有 40320 种

Generate_vaild.py

```
import os

# 创建结果保存目录
result_dir = "generate_out"
os.makedirs(result_dir, exist_ok=True)

def is_valid(list):
    n = len(list)
    for i in range(n):
        for j in range(i + 1, n):
            if abs(list[i] - list[j]) == abs(i - j):
                return False
    return True

# 生成全排列:
def generate_permutations(nums, start_idx, result):
    if start_idx == len(nums) - 1:
        result.append(nums.copy())
        return
    for i in range(start_idx, len(nums)):
        nums[start_idx], nums[i] = nums[i], nums[start_idx]
        generate_permutations(nums, start_idx + 1, result)
        nums[start_idx], nums[i] = nums[i], nums[start_idx]

# 穷举出符合约束的所有排列:
def solutions(numbers):
    all_solutions = []
    all_permutations = []
    generate_permutations(numbers, 0, all_permutations)
    for permutation in all_permutations:
        if is_valid(permutation):
            all_solutions.append(permutation)
    return all_solutions

# 获取结果并保存
if __name__ == "__main__":
    nums = [0, 1, 2, 3, 4, 5, 6, 7]
    results = solutions(nums)
    # 保存结果到文件
    output_file = os.path.join(result_dir, "all_solutions.txt")
    with open(output_file, "w") as f:
        for idx, solution in enumerate(results, 1):
```

```
f.write(f"{solution}\n")
```

```
print(f"Results saved to: {output_file}")
```

```
print(f"Total solutions: {len(results)}")
```

● PS C:\P&T_AI\Exhaustive__non-logical> python generate_vaild.py
Results saved to: generate_out\all_solutions.txt
Total solutions: 92

过滤生成所有有效解

Exhaustive__non-logical > generate_out > ≡ all_solutions.txt									
1	[0, 4, 7, 5, 2, 6, 1, 3]								
2	[0, 5, 7, 2, 6, 3, 1, 4]								
3	[0, 6, 3, 5, 7, 1, 4, 2]								
4	[0, 6, 4, 7, 1, 3, 5, 2]								
5	[1, 3, 5, 7, 2, 0, 6, 4]								
6	[1, 4, 6, 3, 0, 7, 5, 2]								
7	[1, 4, 6, 0, 2, 7, 5, 3]								
8	[1, 5, 0, 6, 3, 7, 2, 4]								
9	[1, 5, 7, 2, 0, 3, 6, 4]								
10	[1, 6, 2, 5, 7, 4, 0, 3]								
11	[1, 6, 4, 7, 0, 3, 5, 2]								
12	[1, 7, 5, 0, 2, 4, 6, 3]								
13	[2, 0, 6, 4, 7, 1, 3, 5]								
14	[2, 4, 1, 7, 0, 6, 3, 5]								
15	[2, 4, 1, 7, 5, 3, 6, 0]								

逻辑编程筛选:

实现步骤:

生成所有排列。

筛选出有效解。

将有效解作为“事实”fact 添加到逻辑编程的关系表 valid_board 中。

最后通过 run 查询将所有事实取出。

```
import itertools
```

```
def check_diagonals(seq):  
    for i in range(8):  
        for j in range(i+1, 8):  
            if abs(i - j) == abs(seq[i] - seq[j]):  
                return False  
    return True
```

```
# 8 个不同数字 0~7 的全排列
```

```
all_perms = list(itertools.permutations(range(8), 8))
```

```
your_solution = (1, 1, 5, 7, 2, 6, 1, 3)
```

```
if your_solution in all_perms and check_diagonals(your_solution):  
    print(f"解 {your_solution} 是有效解")  
else:  
    print(f"解 {your_solution} 不是有效解")
```

举例验证筛选功能:

- PS C:\P&T_AI\Exhaustive__non-logical> python logical_check.py
解 (0, 4, 5, 7, 2, 6, 1, 3) 是有效解
- PS C:\P&T_AI\Exhaustive__non-logical> python logical_check.py
解 (1, 1, 5, 7, 2, 6, 1, 3) 不是有效解
- PS C:\P&T_AI\Exhaustive__non-logical> python logical_check.py
解 (4, 0, 5, 7, 2, 6, 1, 3) 不是有效解

2.逻辑编程（非穷举）

核心逻辑与步骤

生成所有可能排列：

使用 `itertools.permutations(range(8))`生成列索引 `[0,1,2,3,4,5,6,7]`的所有排列，这自动满足了皇后“不在同一行、同一列”的约束。

命令式过滤有效解：

定义函数 `check_diagonals(list)`，通过双重循环检查一个给定的排列是否满足对角线约束。

使用列表推导式 `[p for p in all_perms if check_diagonals(p)]`对所有排列进行过滤，得到所有有效解。

逻辑式声明与查询：

创建一个逻辑编程的关系表 `valid_board`。

将上一步用命令式方法得到的每一个有效解，作为一条“事实” `fact` 加入到这个关系表中。

最后，通过逻辑查询 `run(0, x, valid_board(x))`从关系表中取出所有有效解并计数。

伪代码：

```
// 建立空的关系表
设 valid_board = Relation()

// 定义对角线约束检查函数
函数 check_diagonals(列表)
    for i 从 0 到 7
        for j 从 i+1 到 8
            if |i - j| == |列表[i] - 列表[j]| 则
                返回 假
            结束 if
        结束 for
    结束 for
    返回 真
结束函数

// 穷举生成所有排列
// 生成数字 0-7 的所有 8!种排列
设 “所有排列” = 列表(itertools.permutations(range(8)))

// 筛选满足对角线约束的排列
设 有效排列 = 列表推导式
    对每个“排列 p”在“所有排列” 中
        if check_diagonals(p) 为真
            则 包含 p
```



```

import itertools
from kanren import run, var, fact, Relation
#建立一个空的关系表
valid_board=Relation()

#判断满足对角线约束
def check_diagonals(list):
    for i in range(8):
        for j in range(i+1,8):
            if abs(i-j)==abs(list[i]-list[j]):
                return False
    return True

#穷举法生成所有 8!种排列
all_perms=list(itertools.permutations(range(8)))

#筛选有效解
valid_perms=[p for p in all_perms if check_diagonals(p)]

#逻辑筛选
for p in valid_perms:
    fact(valid_board,p)
#查询结果
x=var()
solutions=run(0,x,valid_board(x))
print(f"共有 {len(solutions)} 种有效解")

```

运行结果:

```

● PS C:\P&T_AI\Exhaustive__non-logical> python pure_logical.py
共有 92 种有效解

```

3. neq 函数

外层函数 `neq(a, b)`:

作用: 它接收两个参数 `a` 和 `b`, 并返回一个新的函数 `goal`。

输入: `a` 和 `b` 可以是逻辑变量 (kanren 的 `var` 对象), 也可以是具体数值。

输出: 一个 `goal` 函数。

内层函数 `goal(s)`:

作用: 接受一个代表当前求解状态 (State) 的参数 `s`, 并检查约束。

核心逻辑: 求解过程中, 将逻辑变量与具体值绑定。绑定的值存储在变量的 `.value` 属性中。

代码检查参数 `a`:

如果 `a` 是一个已被绑定的逻辑变量 (拥有 `value` 属性), 则取出其当前值 `a.value`。

否则 (`a` 是一个普通值, 或者是一个尚未绑定的变量), 则直接使用 `a` 本身。

#neq 函数

```
def neq(a, b):  
    def goal(s):  
        a_val = a.value if hasattr(a, 'value') else a  
        b_val = b.value if hasattr(b, 'value') else b  
        if a_val != b_val:  
            yield s  
    return goal
```

4.带 neq 函数的逻辑编程

伪代码:

```
// 1. 定义不等约束函数
函数 neq(a, b)
    定义 goal(s)
        if a 有属性 'value' 则
            设 a 值 = a.value
        else
            设 a 值 = a
        if b 有属性 'value' 则
            设 b 值 = b.value
        else
            设 b 值 = b
        if a 值 != b 值 则
            yield s
    返回 goal
结束函数

// 定义求解八皇后问题的函数
函数 solve_queens()
    // 创建 8 个逻辑变量
    设 q1, q2, q3, q4, q5, q6, q7, q8 =
        var(), var(), var(), var(), var(), var(), var(), var()
    // 定义可能的取值
    设 值域 = 范围(8)
    // 初始化约束列表
    设 “约束列表” = []
    设 皇后变量列表 = [q1, q2, q3, q4, q5, q6, q7, q8]
    // 添加不同行约束（排列约束）
    for i 从 0 到 7
        for j 从 i+1 到 7
            添加 neq(皇后变量列表[i], 皇后变量列表[j]) 到 “约束列表”
        结束 for
    结束 for

    // 添加对角线约束
    for i 从 0 到 7
        for j 从 i+1 到 7
            添加 neq(绝对值(皇后变量列表[i] - 皇后变量列表[j]), 绝对值(i - j)) 到 “约束列表”
        结束 for
    结束 for
```

```

import itertools
from kanren import run, var, fact, Relation, conde, eq

#neq 函数
def neq(a, b):
    def goal(s):
        a_val = a.value if hasattr(a, 'value') else a
        b_val = b.value if hasattr(b, 'value') else b
        if a_val != b_val:
            yield s
    return goal

def solve_queens():
    q1, q2, q3, q4, q5, q6, q7, q8 =
    var(), var(), var(), var(), var(), var(), var(), var()
    values = range(8)
    # 使用 neq 构建约束
    constraints = []
    queens_vars = [q1, q2, q3, q4, q5, q6, q7, q8]

    # 确保所有变量是不同的（排列）
    for i in range(8):
        for j in range(i+1, 8):
            constraints.append(neq(queens_vars[i], queens_vars[j]))

    #对角线约束
    for i in range(8):
        for j in range(i+1, 8):
            constraints.append(neq(abs(queens_vars[i]-queens_vars[j]), abs(i-j)))

    #运行查询
    return run(0, (q1, q2, q3, q4, q5, q6, q7, q8), *constraints,*[eq(q, values)
                                                                    for q in queens_vars])

# 测试
print("使用 neq 约束方法: ")
solutions = solve_queens()
print(f"共有 {len(solutions)} 种有效解")

```

● PS C:\P&T_AI\Exhaustive__non-logical> python logical_neq.py
 Results saved to: generate_out\all_sequences.txt
 使用neq约束方法:
 共有92种有效解

5. 使用 DFS

核心思路：

使用深度优先搜索（DFS）和回溯法递归求解：

1. 从棋盘第 0 行开始，尝试在 8 列中选择安全位置放置皇后
2. 每次放置前检查：
 - 列冲突：当前列是否已被占用
 - 对角线冲突：是否与已有皇后在同一对角线上
3. 如果当前位置安全，放置皇后并递归处理下一行
4. 如果当前行没有安全位置，回溯到上一行重新选择
5. 当 8 个皇后都成功放置时，保存一个完整解

伪代码：

```
// 1. 初始化存储所有解的列表
设 “所有解” = 空列表

// 2. 定义递归放置皇后的函数
函数 place_queen(当前棋盘状态)
    // 终止条件
    if 长度(当前棋盘状态) == 8 则
        将 当前棋盘状态的副本 添加到 “所有解”
        返回
    结束 if

    // 获取当前要放置的行号
    设 当前行 = 当前棋盘长度

    // 尝试在当前行的每一列放置皇后
    for column 从 0 到 7
        设 is_safe = 真

        // 检查与之前已放置皇后的冲突
        for previous_row 从 0 到 长度(当前棋盘状态)-1
            设 previous_column = 当前棋盘状态[previous_row]

            // 检查列冲突
            if previous_column == column 则
                设 is_safe = 假
                跳出 for
            结束 if

        // 检查对角线冲突
        设 行差 = 绝对值(当前行 - 之前行)
```

```

        设 列差 = 绝对值(列 - 之前列)
        如果 行差 == 列差 则
            设 is_safe = 假
            跳出 for
        结束 if
    结束 for
    // 如果当前位置安全
    if is_safe 为真 则
        将 column 追加到 当前棋盘状态
        调用 place_queen(当前棋盘状态)
        移除 当前棋盘状态 的最后一个元素
    结束 if
结束 for
结束函数

```

```

// 3. 从空棋盘开始搜索
调用 place_queen(空列表)

```

结束

```

def DFS_eight_queens():
    all_solutions = []

    # 递归函数: 尝试在当前棋盘上放置下一个皇后
    # current_board: 列表, 存储已放置皇后的列位置
    def place_queen(current_board):
        # 终止: 如果已经放置了 8 个皇后, 找到一个完整解
        if len(current_board) == 8:
            all_solutions.append(current_board.copy())
            return

        # 当前要放置的行号 (从 0 开始)
        current_row = len(current_board)

        # 尝试在当前行的每一列放置皇后
        for column in range(8):
            is_safe = True

            # 检查与之前已放置的所有皇后是否有冲突
            for previous_row in range(len(current_board)):
                previous_column = current_board[previous_row]

                # 检查 1: 是否在同一列
                if previous_column == column:

```

```

        is_safe = False
        break

    # 检查 2: 是否在同一对角线上 对角线检查: 行差 = 列差的绝对值
    row_difference = abs(current_row - previous_row)
    col_difference = abs(column - previous_column)
    if row_difference == col_difference:
        is_safe = False
        break

    # 如果当前位置安全
    if is_safe:
        current_board.append(column) #做出选择: 放置皇后
        place_queen(current_board) #递归调用: 放置下一个皇后
        current_board.pop() #撤销选择: 回溯, 移除刚放置的皇后

# 从空棋盘开始搜索
place_queen([])
# 输出结果
print(f"找到 {len(all_solutions)} 个解决方案")
#显示前 5 个解决方案
for i, solution in enumerate(all_solutions[:5], 1):
    print(f"方案 {i}: {solution}")
return all_solutions

solutions = DFS_eight_queens()

```

结果:

● PS C:\P&T_AI\DFS_BFS> python dfs.py
找到 92 个解决方案

前5个解决方案 (格式: [行0列, 行1列, ..., 行7列]):

```

方案 1: [0, 4, 7, 5, 2, 6, 1, 3]
方案 2: [0, 5, 7, 2, 6, 3, 1, 4]
方案 3: [0, 6, 3, 5, 7, 1, 4, 2]
方案 4: [0, 6, 4, 7, 1, 3, 5, 2]
方案 5: [1, 3, 5, 7, 2, 0, 6, 4]

```

6. 性能比较与分析

6.1 算法复杂度分析

穷举法复杂度：

时间复杂度： $O(n! \times n^2)$ ，其中 $n=8$

逻辑编程复杂度：

时间复杂度： $O(n! \times n^2)$ ，其中 $n=8$

空间复杂度：中等

逻辑编程(neq 函数)复杂度：

时间复杂度： $O(n!)$ ，其中 $n=8$

最坏时间复杂度： $O(n^n)$

DFS 复杂度：

时间复杂度 $O(n! \times n)$

实际时间复杂度：远小于 $O(n!)$

6.2 方法对比总结

方法	时间复杂度	时间复杂度	优点	缺点
穷举法	$O(n! \times n^2)$	$(O(n!))$	实现简单，逻辑直观	效率最低，内存消耗大
逻辑编程	$O(n! \times n^2)$	$(O(n))$	代码简洁，引入声明式思想	效率低，未发挥逻辑编程优势
逻辑编程 (neq 函数)	$O(n!)$	$(O(n))$	代码最简洁，声明性强，符合人类思维	运行效率依赖引擎，通常低于优化回溯算法
DFS	远小于 $O(n!)$	$(O(n))$	内存效率高，找到首解快，最实用	实现稍复杂于穷举法

7. 调试过程与实验总结

调试过程：

问题：

`TypeError: var() - int`

原因分析：

命令式编程思维与逻辑编程范式的冲突。在约束构建阶段，直接对可能未赋值的逻辑变量（var 对象）使用了 Python 的原生数学运算符（如 `-`，`abs()`），导致 Python 解释器在运行时崩溃。

解决核心：

必须将数学运算的判断逻辑，封装成一个 kanren 能够理解的、延迟执行的“目标”（goal）函数，而不是在外部直接计算。这个目标函数会在 kanren 引擎的控制下，仅在相关逻辑变量被具体化（赋值）后才执行内部的判断逻辑。

使用 `conso`，`lall`，`conde` 和递归/迭代来定义“不等于”关系。

实验总结：

先采用命令式编程，通过穷举法生成所有排列并逐个验证，逻辑直观但效率最低。

深度优先搜索（DFS）回溯算法则在效率上取得巨大提升，因为它能提前剪枝，避免无效搜索。

这两种命令式方法需要我们为计算机指定每一步的具体步骤。

接着完成逻辑编程的两种实现。

第一种是“披着逻辑外衣的命令式”，本质是先用命令式方法求出所有解，再作为“事实”填入知识库，并未体现逻辑编程的“推导”优势。

第二种方法是通过自定义 `neq` 函数来声明约束（如“皇后不能在同一列”），这更符合逻辑编程的声明式思想，即只告诉计算机“规则”，由引擎来推导出所有满足条件的解。

最终，通过对算法复杂度的分析，实验总结认为：虽然 DFS 是最高效实用的命令式解法，但通过 `neq` 函数构建约束的逻辑编程方法在代码简洁性和“只描述问题、不描述过程”的抽象层级上最具优势，更贴近人类解决问题的逻辑思维。

8. 可改进的方向

已经实现的深度优先搜索（DFS）算法能在可接受时间内求解八皇后问题，但在算法设计与实现上仍存在进一步优化的空间。

同时，广度优先搜索（BFS）作为一种替代策略，也可以在实验中进行探索与比较。

8.1 深度优先搜索（DFS）的优化空间

目前 DFS 实现采用递归回溯，逐行放置皇后，并在每次放置前检查与已放置皇后的冲突。在以下方面仍可改进：

对称性剪枝：八皇后问题的解具有对称性。

可以通过引入对称性检测，在搜索过程中避免生成等价解，从而减少搜索空间。

例如，强制第一个皇后放置在棋盘的一半区域内，即可将解的数量减少约一半。

启发式排序：在当前行的列选择上，可引入启发式策略（优先选择冲突最少的列），使算法更早找到有效解，提升剪枝效率。这在处理更大规模（如 N 皇后）问题时尤为显著。

8.2 补充广度优先搜索（BFS）

广度优先搜索（BFS）在八皇后问题中虽不如 DFS 常用，但作为一种系统性的搜索策略，具有研究价值。BFS 按层扩展状态空间，逐行放置皇后，其特点如下：

层次遍历：BFS 从空棋盘开始，逐层放置皇后，直到所有皇后放置完毕。

每层对应一个行号，所有状态按行数分层存储。

状态去重：BFS 可结合哈希表记录已访问状态，避免重复扩展，确保搜索的完备性。

内存消耗大：与 DFS 相比，BFS 需要维护当前层的所有状态，空间复杂度较高。

对于 $n=8$ ，BFS 尚可接受，但随 n 增大，状态数呈指数增长，实用性受限。

适用场景：BFS 更适合用于寻找最短路径或最小步骤数的问题，而在八皇后问题中，DFS 因其更低的内存占用和更快的解生成能力，通常更为高效。

8.3 改进方向

为进一步提升实验的完整性与算法比较的深度，可考虑以下方向：

实现基于位运算的 DFS：展示如何通过位运算优化冲突检测与回溯过程，提升求解效率。

实现 BFS 版本：对比 DFS 与 BFS 在时间、空间上的差异，分析二者在不同规模问题下的适用性。

引入性能测试：对不同方法（穷举法、逻辑编程、DFS、BFS）进行运行时间与内存占用的定量比较，使用 `time` 与 `memory_profiler` 等工具进行测量，用数据支撑算法优劣分析。

扩展到 N 皇后问题：将当前实现推广到 N 皇后，观察各算法在 $n=10, 12, 14$ 等更大规模下的表现差异，进一步验证算法复杂度分析的正确性。